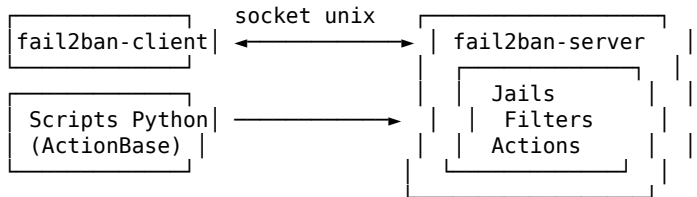


BIBLE FAIL2BAN (PYTHON) – RÉFÉRENCE EXHAUSTIVE

LÉGENDE : [cli] = fail2ban-client | [cfg] = configuration fichiers
[jail] = jails | [flt] = filtres/regex | [act] = actions
[py] = actions Python (ActionBase) | [sock] = socket API Python
[db] = base de données | [cmd] = commandes complètes

ARCHITECTURE FAIL2BAN :



fail2ban-server ← NE JAMAIS appeler directement (toujours via fail2ban-client)

[cfg] Installation et structure des fichiers

DESCRIPTION : Daemon Python de prévention d'intrusion basé sur l'analyse de logs
POURQUOI : Bannir automatiquement les IPs qui déclenchent trop d'erreurs d'auth.
CONTEXTE : Serveurs Linux, SSH, Apache, Nginx, Postfix, toute app avec des logs.

INSTALLATION :

sudo apt install fail2ban	Debian/Ubuntu
sudo dnf install fail2ban	Fedora/RHEL
sudo yum install fail2ban	CentOS
sudo pacman -S fail2ban	Arch Linux
pip install fail2ban	(rare, préférer le package système)

STRUCTURE DES FICHIERS :

/etc/fail2ban/	
├─ fail2ban.conf	Config principale (socket, PID, logs)
├─ fail2ban.local	Surcharge locale (ne jamais éditer .conf)
├─ jail.conf	Jails par défaut (NE PAS ÉDITER)
├─ jail.local	□ Vos surcharges de jails
├─ jail.d/	□ Fichiers .conf et .local par jail
│ ├─ defaults-debian.conf	
│ └─ ma-jail.conf	
├─ filter.d/	Filtres (regex de détection)
│ ├─ sshd.conf	
│ ├─ apache-auth.conf	
│ └─ mon-filtre.conf	□ Vos filtres personnalisés
├─ action.d/	Actions (ban/unban)
│ ├─ iptables.conf	
│ ├─ iptables-multiport.conf	
│ ├─ sendmail.conf	
│ ├─ cloudflare.conf	
│ ├─ smtp.py	Action Python native
│ └─ mon-action.conf	□ Vos actions personnalisées

PRIORITÉ DE LECTURE (du moins au plus prioritaire) :

jail.conf → jail.d/*.conf → jail.local → jail.d/*.local

FICHIERS RUNTIME :

/var/run/fail2ban/fail2ban.sock	Socket Unix de communication
/var/run/fail2ban/fail2ban.pid	PID du daemon
/var/log/fail2ban.log	Fichier de log principal
/var/lib/fail2ban/fail2ban.sqlite3	Base de données des bans

SERVICE SYSTEMD :

sudo systemctl start fail2ban	
sudo systemctl stop fail2ban	
sudo systemctl restart fail2ban	
sudo systemctl reload fail2ban	Recharge la config à chaud
sudo systemctl status fail2ban	
sudo systemctl enable fail2ban	Démarrage automatique

[cfg] fail2ban.conf / fail2ban.local – configuration principale

DESCRIPTION : Paramètres globaux du daemon fail2ban
 POURQUOI : Contrôler le socket, le PID, le niveau de log et la base de données.
 CONTEXTE : Créer /etc/fail2ban/fail2ban.local pour surcharger fail2ban.conf.

PARAMÈTRES PRINCIPAUX :

[Definition]	
loglevel = INFO	CRITICAL, ERROR, WARNING, NOTICE, INFO, DEBUG, TRACEDEBUG, HEAVYDEBUG
logtarget = /var/log/fail2ban.log	Fichier de log, STDOUT, STDERR, SYSLOG, SYSTEMD-JOURNAL
syslogsocket = auto	Socket syslog
socket = /var/run/fail2ban/fail2ban.sock	Socket Unix (communication client/server)
pidfile = /var/run/fail2ban/fail2ban.pid	Fichier PID
dbfile = /var/lib/fail2ban/fail2ban.sqlite3	Base SQLite des bans persistants
dbpurgeage = 86400	Durée de conservation en base (secondes)
dbmaxmatches = 10	Nombre max de matches par ticket en DB

[jail] jail.local - configuration des jails

DESCRIPTION : Blocs de configuration définissant la surveillance d'un service
 POURQUOI : Chaque jail surveille un service spécifique (SSH, Apache, Nginx...)
 CONTEXTE : Copier jail.conf en jail.local, modifier uniquement ce qui est nécessaire.

SECTION [DEFAULT] :

[DEFAULT]	
bantime = 10m	Durée du ban (s, m, h, d, w)
bantime = -1	Ban permanent
bantime = 1h	1 heure
findtime = 10m	Fenêtre de détection des tentatives
maxretry = 5	Tentatives avant ban dans findtime
maxmatches = %(maxretry)s	Matches max stockés en mémoire
ignoreself = true	Ignore les IPs locales du serveur
ignoreip = 127.0.0.1/8 ::1	IPs jamais bannies (espace/virgule)
ignoreip = 127.0.0.1/8 ::1 192.168.1.0/24 203.0.113.10	
action = %(action_)s	Action par défaut (iptables)
# Raccourcis d'action :	
# action_ = ban seulement	
# action_mw = ban + email simple	
# action_mwl = ban + email avec logs	
# action_cf = Cloudflare	
banaction = iptables-multiport	Action de ban (fichier dans action.d/)
banaction_allports = iptables-allports	Ban sur tous les ports
protocol = tcp	Protocole (tcp, udp, all)
chain = INPUT	Chaîne iptables
port = ssh	Port(s) surveillé(s)
logencoding = auto	Encodage des logs (auto, utf-8, ascii)
enabled = false	Activer/désactiver le jail par défaut
mode = normal	Mode de filtrage (normal, ddos, extra...)
backend = auto	Backend: auto, pyinotify, gamin, polling, systemd
usedns = warn	warn, yes, no, raw (résolution DNS des IPs)
destemail = root@localhost	Email destinataire (pour actions _mw_)
sender = fail2ban@localhost	Email expéditeur
mta = sendmail	Agent mail (sendmail, mail, postfix)

STRUCTURE D'UN JAIL :

[sshd]	
enabled = true	Active ce jail
port = ssh,22	Port(s) à protéger
filter = sshd	Fichier filter.d/sshd.conf
logpath = /var/log/auth.log	Fichier(s) de log à surveiller
logpath = %(sshd_log)s	Variable prédéfinie (Debian)
= /var/log/secure	(RHEL/CentOS)
maxretry = 3	Surcharge du DEFAULT
bantime = 1h	
findtime = 15m	
backend = %(sshd_backend)s	

— JAILS COURANTS PRÉCONFIGURÉS —

[sshd]	SSH
--------	-----

[apache-auth]	Authentification Apache
[apache-badbots]	Bots Apache malveillants
[apache-noscript]	Scripts inexistants
[apache-overflows]	Overflows Apache
[nginx-http-auth]	Auth HTTP Nginx
[nginx-botsearch]	Bots Nginx
[nginx-limit-req]	Rate limiting Nginx
[postfix]	Postfix SMTP
[postfix-sasl]	Auth SASL Postfix
[dovecot]	Dovecot IMAP/POP3
[proftpd]	ProFTPD
[vsftpd]	vsftpd
[mysqld-auth]	MySQL
[recidive]	Bans récidivistes (ban multi-jails)
[pam-generic]	PAM générique
[phpmyadmin-syslog]	phpMyAdmin

— EXEMPLES DE JAILS PERSONNALISÉS —

```
# SSH renforcé
[sshd]
enabled = true
maxretry = 3
bantime = 24h
findtime = 1h

# Application web personnalisée
[mon-app-web]
enabled = true
port = http,https,8080
filter = mon-app-web
logpath = /var/log/mon-app/access.log
maxretry = 10
bantime = 30m
findtime = 5m
action = iptables-multiport[name=mon-app, port="http,https,8080"]

# Ban permanent des récidivistes
[recidive]
enabled = true
logpath = /var/log/fail2ban.log
banaction = iptables-allports
bantime = -1
findtime = 1d
maxretry = 5

# Protection avec notification email
[sshd-with-mail]
enabled = true
action = %(action_mwl)s
destemail = admin@exemple.com
sender = fail2ban@monserveur.com
```

```
[flt] Filtres - filter.d/*.conf
```

DESCRIPTION : Expressions régulières Python pour détecter les tentatives d'attaque
 POURQUOI : Définit CE QUI constitue une tentative dans les logs.
 CONTEXTE : Chaque jail référence un filtre dans filter.d/.

STRUCTURE D'UN FICHIER FILTRE :

```
[INCLUDES]
before = common.conf          Inclure des définitions communes

[Definition]
_daemon = sshd                Variable locale

failregex = ^%(__prefix_line)s(?:error: PAM: )?[aA]uthentication(?:failure|error|failed)
            for .* from <HOST>\s*$
            ^%(__prefix_line)s Invalid user .* from <HOST>\s*$

ignoreregex =                  Lignes à ignorer (regex)

journalmatch = _SYSTEMD_UNIT=sshd.service  Pour les backends systemd
```

datepattern = {^LN-BEG}%Y-%m-%d %H:%M:%S Format de date dans les logs

— VARIABLES SPÉCIALES —

<HOST>	Remplacé par l'IP capturée (IPv4 ou IPv6)
<ADDR>	Alias de <HOST>
<SUBNET>	Sous-réseau
<F-ID>	ID unique (pour ban groupé)
<F-MLFFORGET>	Forcer l'oubli des lignes
%(__prefix_line)s	Préfixe de ligne (depuis common.conf)
%(daemon_name)s	Nom du daemon
{EPOCH}	Timestamp epoch dans les logs

— VARIABLES COMMUNES (common.conf) —

%(__prefix_line)s	→ timestamp + hostname
%(__bsd_syslog_verbose)s	
%(__line_prefix)s	

— TEST D'UN FILTRE —

fail2ban-regex FICHIER_LOG FILTRE	Test en live
fail2ban-regex /var/log/auth.log sshd	Test du filtre sshd sur auth.log
fail2ban-regex /var/log/nginx/error.log nginx-http-auth	
fail2ban-regex /var/log/mon-app.log /etc/fail2ban/filter.d/mon-filtre.conf	
fail2ban-regex "ligne de log" "regex_failregex" -v Verbose	
fail2ban-regex --print-all-matched /var/log/auth.log sshd	Toutes les correspondances

EXEMPLE DE FILTRE PERSONNALISÉ :

```
# /etc/fail2ban/filter.d/mon-app.conf
[INCLUDES]
before = common.conf
```

[Definition]

```
failregex = ^.*\[error\].*authentication failed.*from <HOST>.*$
            ^.*Failed login attempt from <HOST>.*$
            ^<HOST> .* "POST /wp-login.php.*" 200.*$
```

```
ignoreregex = ^.*bot.*$
```

```
            ^.*googlebot.*$
```

```
datepattern = %Y-%m-%d %H:%M:%S
```

[act] Actions — action.d/*.conf

DESCRIPTION : Définit CE QUI est fait quand une IP est bannie/débannie
POURQUOI : Ajouter une règle firewall, envoyer un email, appeler une API...
CONTEXTE : Fichiers .conf (commandes shell) ou .py (actions Python).

STRUCTURE D'UN FICHIER ACTION .conf :

```
[INCLUDES]
before = iptables-common.conf                    Inclure des définitions communes
```

[Definition]

actionstart = <commande_shell>	Exécutée au démarrage du jail
actionstop = <commande_shell>	Exécutée à l'arrêt du jail
actioncheck = <commande_shell>	Vérifie que l'action est opérationnelle
actionban = <commande_shell>	Exécutée lors d'un ban
actionunban = <commande_shell>	Exécutée lors d'un déban

[Init]

name = default	Valeur par défaut de <name>
port = ssh	Valeur par défaut de <port>
chain = INPUT	
bantime = 600	

— VARIABLES DISPONIBLES DANS LES ACTIONS —

<ip>	IP bannie
<port>	Port(s) du jail
<protocol>	Protocole (tcp, udp...)
<chain>	Chaîne iptables
<name>	Nom du jail
<bantime>	Durée du ban en secondes
<failures>	Nombre de tentatives détectées
<time>	Timestamp du ban
<matches>	Lignes de log correspondantes

<ipmatches>	Toutes les correspondances de cette IP
<ipjailmatches>	Correspondances IP dans ce jail
<cidr>	CIDR de l'IP bannie

— ACTIONS PRÉCONFIGURÉES IMPORTANTES —

iptables	iptables simple
iptables-multiport	iptables multi-ports (recommandé) ☐
iptables-allports	Bannit tous les ports
iptables-ipset	Utilise ipset (performant pour gros volumes)
ip6tables-multiport	IPv6 multi-ports
nftables-multiport	nftables (moderne, remplace iptables)
ufw	UFW (Ubuntu)
firewallcmd-ipset	firewalld + ipset (RHEL/CentOS)
pf	PF (FreeBSD/macOS)
hostsdeny	/etc/hosts.deny (TCP Wrappers)
sendmail-ban	Email à l'administrateur sur ban
sendmail-whois	Email + whois de l'IP bannie
sendmail-whois-lines	Email + whois + lignes de log
cloudflare	API Cloudflare (bloquer sur CDN)
abuseipdb	API AbuseIPDB (signaler l'IP)
nginx-block-map	Nginx block map dynamique
route	Blackhole routing (null route)
badips	Service badips.com

EXEMPLE D'ACTION PERSONNALISÉE (ban via API REST) :

```
# /etc/fail2ban/action.d/mon-api-ban.conf
[Definition]
actionban = curl -s -X POST https://mon-api.exemple.com/ban \
            -H "Authorization: Bearer MONTOKEN" \
            -d '{"ip": "<ip>", "jail": "<name>", "bantime": <bantime>}'

actionunban = curl -s -X DELETE https://mon-api.exemple.com/ban/<ip> \
              -H "Authorization: Bearer MONTOKEN"

[Init]
```

[py] Actions Python — ActionBase

DESCRIPTION : API Python pour créer des actions fail2ban en Python pur
 POURQUOI : Logique complexe impossible en shell (API HTTP, bases de données...)
 CONTEXTE : Fichiers .py dans action.d/, référencés dans jail.local avec JSONKWARGS.

— CLASSE DE BASE —

```
from fail2ban.server.actions import ActionBase, CallingMap
```

```
class MonAction(ActionBase):
    """Action fail2ban personnalisée en Python."""

    def __init__(self, jail, name, **kwargs):
        super().__init__(jail, name)
        # Vos paramètres de config (depuis JSONKWARGS)

    def start(self):
        """Appelé au démarrage du jail."""
        pass

    def stop(self):
        """Appelé à l'arrêt du jail."""
        pass

    def ban(self, aInfo):
        """Appelé lors d'un ban."""
        # aInfo contient les infos du ban
        ip = aInfo.get("ip")
        failures = aInfo.get("failures")
        matches = aInfo.get("matches")
        bantime = aInfo.get("bantime")

    def unban(self, aInfo):
        """Appelé lors d'un déban."""
        ip = aInfo.get("ip")
```

— DICTIONNAIRE aInfo —

aInfo["ip"]	IP bannie (str)
aInfo["failures"]	Nombre de tentatives (int)
aInfo["matches"]	Lignes de log (list de str)
aInfo["ipmatches"]	Toutes les correspondances IP (list)
aInfo["ipjailmatches"]	Correspondances IP dans ce jail (list)
aInfo["bantime"]	Durée du ban en secondes (int)
aInfo["time"]	Timestamp du ban (float)
aInfo["jail"]	Nom du jail (str)

— CallingMap —

```
# Dict spécial qui évalue les callables au moment de l'accès
from fail2ban.server.actions import CallingMap
import socket

self.info = CallingMap(
    jailname = self._jail.name,          Valeur statique
    hostname = socket.gethostname(),     Callable – évalué à l'accès
    bantime  = lambda: self._jail.actions.getBanTime(),
)
# Accès : self.info["hostname"] appelle socket.gethostname()
```

— LOGGING DANS UNE ACTION PYTHON —

```
# self._logSys est un logger Python standard disponible dans ActionBase
self._logSys.debug("Message de débogage")
self._logSys.info("Informations")
self._logSys.warning("Avertissement")
self._logSys.error("Erreur : %s", str(e))
self._logSys.critical("Critique")
```

— RÉFÉRENCER UNE ACTION PYTHON DANS JAIL.LOCAL —

```
[mon-jail]
action = mon-action[kwarg1="valeur1", kwarg2="valeur2"]
# Pour une action Python :
action = mon-action.py[kwarg1="valeur1"]
# Format complet :
set <JAIL> addaction <NOM> /etc/fail2ban/action.d/mon-action.py \
    '{"kwarg1": "valeur1", "kwarg2": "valeur2"}'
```

— norestored —

```
self.norestored = 1                Ne pas exécuter ban() pour les bans restaurés
# Utile pour les actions email (ne pas renvoyer d'emails au redémarrage)
```

EXEMPLE COMPLET : Action Python qui appelle une API REST

```
# /etc/fail2ban/action.d/api-ban.py
import requests
from fail2ban.server.actions import ActionBase

class Action(ActionBase):
    def __init__(self, jail, name, api_url, api_token):
        super().__init__(jail, name)
        self.api_url = api_url
        self.api_token = api_token
        self.norestored = 1

    def ban(self, aInfo):
        ip = aInfo.get("ip")
        try:
            r = requests.post(
                f"{self.api_url}/ban",
                headers={"Authorization": f"Bearer {self.api_token}"},
                json={"ip": ip, "jail": self._jail.name,
                    "failures": aInfo.get("failures")},
                timeout=10
            )
            self._logSys.info("Banni %s via API : %s", ip, r.status_code)
        except Exception as e:
            self._logSys.error("Échec ban API pour %s : %s", ip, e)

    def unban(self, aInfo):
        ip = aInfo.get("ip")
        try:
            requests.delete(
                f"{self.api_url}/ban/{ip}",
                headers={"Authorization": f"Bearer {self.api_token}"},
                timeout=10
            )
```

```

    )
    except Exception as e:
        self._logSys.error("Échec unban API pour %s : %s", ip, e)

# Dans jail.local :
# action = api-ban.py[api_url="https://mon-api.com", api_token="TOKEN"]

```

[sock] Interaction Python via socket Unix – API programmation

DESCRIPTION : Communiquer avec fail2ban-server depuis Python via le socket Unix
 POURQUOI : Automatiser les bans/débans et requêtes de statut depuis du code Python.
 CONTEXTE : Scripts de monitoring, tableaux de bord, intégrations tierces.

MÉTHODE 1 : subprocess (la plus simple et la plus robuste)

```

import subprocess

def f2b(args):
    """Lance fail2ban-client avec les arguments donnés et retourne la sortie."""
    result = subprocess.run(
        ["fail2ban-client"] + args,
        capture_output=True,
        text=True
    )
    if result.returncode != 0:
        raise RuntimeError(f"fail2ban-client {' '.join(args)}: {result.stderr}")
    return result.stdout.strip()

# Ping du serveur
f2b(["ping"]) # → "Server replied: pong"

# Lister les jails actifs
f2b(["status"]) # → "Status\n|- Number of jail: 2\n`- Jail list: sshd, nginx"

# Statut d'un jail
f2b(["status", "sshd"])

# Bannir une IP
f2b(["set", "sshd", "banip", "1.2.3.4"])

# Débannir une IP
f2b(["set", "sshd", "unbanip", "1.2.3.4"])

# Ajouter une IP à la liste d'ignorance
f2b(["set", "sshd", "addignoreip", "192.168.1.0/24"])

```

MÉTHODE 2 : socket Unix direct (bas niveau)

```

import socket
import pickle
import struct

SOCKET_PATH = "/var/run/fail2ban/fail2ban.sock"

def envoyer_commande(commande_liste):
    """Envoie une commande au serveur fail2ban via socket Unix."""
    with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as sock:
        sock.connect(SOCKET_PATH)
        # Sérialiser la commande avec pickle
        data = pickle.dumps(commande_liste)
        # Envoyer la taille puis les données
        sock.sendall(struct.pack(">i", len(data)) + data)
        # Recevoir la réponse
        size_data = sock.recv(4)
        size = struct.unpack(">i", size_data)[0]
        response = sock.recv(size)
        return pickle.loads(response)

# Exemples d'utilisation
envoyer_commande(["ping"])
envoyer_commande(["status"])
envoyer_commande(["status", "sshd"])
envoyer_commande(["set", "sshd", "banip", "1.2.3.4"])

```

MÉTHODE 3 : CommunicatingProcess (API interne fail2ban)

```
# Utilise les modules internes de fail2ban (nécessite fail2ban installé)
import sys
sys.path.insert(0, "/usr/share/fail2ban")    Ou chemin d'installation

from fail2ban.client.csocket import CSocket

socket_path = "/var/run/fail2ban/fail2ban.sock"
csock = CSocket(socket_path)

# Envoyer une commande
response = csock.send(["status"])
response = csock.send(["status", "sshd"])
response = csock.send(["set", "sshd", "banip", "10.0.0.1"])
response = csock.send(["set", "sshd", "unbanip", "10.0.0.1"])
```

CLASSE UTILITAIRE COMPLÈTE :

```
import subprocess, re

class Fail2BanClient:
    """Interface Python haut niveau vers fail2ban-client."""

    def __init__(self, socket=None):
        self._base = ["fail2ban-client"]
        if socket:
            self._base += ["-s", socket]

    def _run(self, *args):
        r = subprocess.run(self._base + list(args),
                           capture_output=True, text=True)
        if r.returncode != 0:
            raise RuntimeError(r.stderr.strip())
        return r.stdout.strip()

    def ping(self):
        return "pong" in self._run("ping")

    def reload(self, jail=None):
        if jail:
            return self._run("reload", jail)
        return self._run("reload")

    def jails(self):
        """Retourne la liste des jails actifs."""
        sortie = self._run("status")
        match = re.search(r"Jail list:\s*(.+)", sortie)
        if match:
            return [j.strip() for j in match.group(1).split(",") if j.strip()]
        return []

    def status(self, jail):
        """Retourne le statut d'un jail sous forme de dict brut."""
        return self._run("status", jail)

    def banned_ips(self, jail):
        """Retourne la liste des IPs bannies dans un jail."""
        sortie = self._run("status", jail)
        match = re.search(r"Banned IP list:\s*(.+)", sortie)
        if match:
            return match.group(1).strip().split()
        return []

    def ban(self, jail, ip):
        return self._run("set", jail, "banip", ip)

    def unban(self, jail, ip):
        return self._run("set", jail, "unbanip", ip)

    def unban_all(self, jail):
        return self._run("set", jail, "unbanip", "--all")

    def add_ignore(self, jail, ip):
        return self._run("set", jail, "addignoreip", ip)

    def del_ignore(self, jail, ip):
        return self._run("set", jail, "delignoreip", ip)
```



```

def get_ignore(self, jail):
    return self._run("get", jail, "ignoreip")

def set_bantime(self, jail, seconds):
    return self._run("set", jail, "bantime", str(seconds))

def set_maxretry(self, jail, n):
    return self._run("set", jail, "maxretry", str(n))

def get_bantime(self, jail):
    return self._run("get", jail, "bantime")

def get_maxretry(self, jail):
    return self._run("get", jail, "maxretry")

def start_jail(self, jail):
    return self._run("start", jail)

def stop_jail(self, jail):
    return self._run("stop", jail)

# Utilisation :
f2b = Fail2BanClient()
if f2b.ping():
    print("Jails :", f2b.jails())
    print("IPs bannies SSH :", f2b.banned_ips("sshd"))
    f2b.ban("sshd", "1.2.3.4")
    f2b.unban("sshd", "1.2.3.4")

```

```
[cmd] fail2ban-client - commandes complètes
```

DESCRIPTION : Référence complète de toutes les commandes fail2ban-client
 POURQUOI : Administrer fail2ban depuis le terminal.
 CONTEXTE : Administration, débogage, scripts shell, cron.

— SERVEUR —

fail2ban-client start	Démarre le serveur fail2ban
fail2ban-client stop	Arrête le serveur
fail2ban-client restart	Redémarre
fail2ban-client reload	Recharge la config (sans redémarrer)
fail2ban-client reload --restart	Recharge + redémarre les jails affectés
fail2ban-client reload --unban	Recharge + débannit les IPs
fail2ban-client reload JAIL	Recharge un jail spécifique
fail2ban-client ping	Vérifie que le serveur répond
fail2ban-client status	Statut global + liste des jails
fail2ban-client version	Version de fail2ban
fail2ban-client --help	Aide

— OPTIONS DE FAIL2BAN-CLIENT —

fail2ban-client -v	Verbose (+1 niveau par -v)
fail2ban-client -vvv	Très verbose
fail2ban-client -q	Silencieux
fail2ban-client -s /chemin/vers/socket	Socket custom
fail2ban-client -x	Force le démarrage (ignore le socket existant)

— CONTRÔLE DES JAILS —

fail2ban-client status	Liste tous les jails
fail2ban-client status JAIL	Statut détaillé d'un jail
fail2ban-client status JAIL --flavor short	Statut court
fail2ban-client status JAIL --flavor long	Statut long (avec matches)
fail2ban-client start JAIL	Démarre un jail
fail2ban-client stop JAIL	Arrête et supprime un jail
fail2ban-client reload JAIL	Recharge un jail

— BAN / DÉBAN —

fail2ban-client set JAIL banip IP	Bannit une IP dans un jail
fail2ban-client set JAIL banip IP1 IP2	Bannit plusieurs IPs
fail2ban-client set JAIL unbanip IP	Débannit une IP
fail2ban-client set JAIL unbanip --all	Débannit toutes les IPs du jail
fail2ban-client unban IP	Débannit dans TOUS les jails (v0.10+)
fail2ban-client unban --all	Débannit tout dans tous les jails

— LISTE D'IGNORANCE —

fail2ban-client	set	JAIL addignoreip IP	Ajoute une IP à ignorer
fail2ban-client	set	JAIL delignoreip IP	Retire une IP de la liste
fail2ban-client	get	JAIL ignoreip	Liste les IPs ignorées

— CONFIGURATION DES JAILS À LA VOLÉE —

fail2ban-client	set	JAIL idle on	Met le jail en pause
fail2ban-client	set	JAIL idle off	Reprend le jail
fail2ban-client	set	JAIL maxretry N	Modifie maxretry
fail2ban-client	set	JAIL bantime N	Modifie bantime (secondes)
fail2ban-client	set	JAIL findtime N	Modifie findtime
fail2ban-client	set	JAIL maxlines N	Modifie le buffer de lignes
fail2ban-client	get	JAIL maxretry	Lit maxretry actuel
fail2ban-client	get	JAIL bantime	Lit bantime actuel
fail2ban-client	get	JAIL findtime	Lit findtime actuel
fail2ban-client	get	JAIL logpath	Liste des fichiers surveillés
fail2ban-client	get	JAIL ignoreip	IPs ignorées
fail2ban-client	get	JAIL actions	Liste des actions du jail
fail2ban-client	get	JAIL loglevel	Niveau de log du jail
fail2ban-client	set	JAIL addlogpath FICHIER	Ajoute un fichier de log à surveiller
fail2ban-client	set	JAIL dellogpath FICHIER	Retire un fichier de log
fail2ban-client	set	JAIL ignoreself true	Ignore les IPs locales
fail2ban-client	set	JAIL ignoreself false	

— CONFIGURATION GLOBALE À LA VOLÉE —

fail2ban-client	set	loglevel DEBUG	Niveau de log global
fail2ban-client	set	loglevel INFO	
fail2ban-client	get	loglevel	Niveau de log actuel
fail2ban-client	set	logtarget /var/log/fail2ban.log	
fail2ban-client	get	logtarget	
fail2ban-client	set	dbfile /var/lib/fail2ban/fail2ban.sqlite3	
fail2ban-client	get	dbfile	
fail2ban-client	set	dbpurgeage 86400	Purge la DB après 86400s
fail2ban-client	get	dbpurgeage	

— ACTIONS À LA VOLÉE —

fail2ban-client	set	JAIL addaction NOM	Ajoute une action Command
fail2ban-client	set	JAIL addaction NOM FICHIER.py JSON	Action Python + kwargs
fail2ban-client	set	JAIL delaction NOM	Supprime une action
fail2ban-client	set	JAIL action NOM actionban CMD	
fail2ban-client	set	JAIL action NOM actionunban CMD	
fail2ban-client	set	JAIL action NOM actionstart CMD	
fail2ban-client	set	JAIL action NOM actionstop CMD	
fail2ban-client	get	JAIL action NOM actionban	
fail2ban-client	get	JAIL action NOM timeout	

— BASE DE DONNÉES —

fail2ban-client	get	dbfile	Chemin de la DB SQLite
fail2ban-client	get	dbpurgeage	Âge de purge en secondes
fail2ban-client	set	dbpurgeage 604800	Purge après 7 jours
fail2ban-client	set	dbmaxmatches 10	Max matches par ticket en DB

— OUTILS COMPLÉMENTAIRES —

fail2ban-regex	LOGFILE FILTERFILE	Test d'un filtre sur un fichier de log
fail2ban-regex	/var/log/auth.log sshd -v	Verbose
fail2ban-regex	/var/log/auth.log sshd --print-all-matched	
fail2ban-regex	/var/log/auth.log sshd --print-no-missed	
fail2ban-regex	"texte de log" "regex"	Test une ligne unique
fail2ban-server	-b	(Ne pas utiliser directement)

— EXEMPLES DE WORKFLOWS COURANTS —

```
# Vérifier les jails actifs
sudo fail2ban-client status

# Inspecter un jail SSH
sudo fail2ban-client status sshd

# Bannir manuellement une IP dans tous les jails SSH
sudo fail2ban-client set sshd banip 192.168.1.100

# Débannir une IP partout (v0.10+)
sudo fail2ban-client unban 192.168.1.100
```

```
# Ajouter son IP en whitelist SSH
sudo fail2ban-client set sshd addignoreip 203.0.113.5

# Recharger la config sans perdre les bans actifs
sudo fail2ban-client reload

# Tester un filtre personnalisé
fail2ban-regex /var/log/nginx/error.log /etc/fail2ban/filter.d/mon-filtre.conf -v
```

[db] Base de données SQLite – /var/lib/fail2ban/fail2ban.sqlite3
--

```
DESCRIPTION : Base de données SQLite persistant les bans entre les redémarrages
POURQUOI : Restaurer les bans après un redémarrage du serveur.
CONTEXTE : Lecture directe pour monitoring, stats, intégrations.
```

TABLES PRINCIPALES :

bans	Historique des bans
logs	Fichiers de log surveillés
jail	Jails connus

REQUÊTES SQL UTILES :

```
-- Tous les bans actifs
SELECT jail, ip, timeofban, bantime
FROM bans
WHERE timeofban + bantime > strftime('%s', 'now')
ORDER BY timeofban DESC;

-- IPs les plus bannies
SELECT ip, COUNT(*) as nb_bans
FROM bans
GROUP BY ip
ORDER BY nb_bans DESC
LIMIT 20;

-- Bans des dernières 24h
SELECT jail, ip, datetime(timeofban, 'unixepoch') as date_ban, bantime
FROM bans
WHERE timeofban > strftime('%s', 'now') - 86400
ORDER BY timeofban DESC;

-- Bans dans un jail spécifique
SELECT ip, datetime(timeofban, 'unixepoch') as date_ban, matches
FROM bans
WHERE jail = 'sshd'
ORDER BY timeofban DESC;
```

ACCÈS PYTHON :

```
import sqlite3
from datetime import datetime

DB_PATH = "/var/lib/fail2ban/fail2ban.sqlite3"

with sqlite3.connect(DB_PATH) as conn:
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    # Bans actifs
    cursor.execute("""
        SELECT jail, ip, timeofban, bantime
        FROM bans
        WHERE timeofban + bantime > strftime('%s', 'now')
        ORDER BY timeofban DESC
    """)
    for row in cursor.fetchall():
        ts = datetime.fromtimestamp(row["timeofban"])
        print(f"{row['jail']}: {row['ip']} (banni le {ts})")

    # Top IPs agressives
    cursor.execute("""
        SELECT ip, COUNT(*) as total
        FROM bans GROUP BY ip
        ORDER BY total DESC LIMIT 10
    """)
```

```

"""
for row in cursor.fetchall():
    print(f"{row['ip']}: {row['total']} bans")

```

[util] Patterns et recettes Python

— MONITORING AUTOMATIQUE DES BANS —

```

import subprocess, re, time

def get_banned_ips(jail):
    r = subprocess.run(["fail2ban-client", "status", jail],
                       capture_output=True, text=True)
    m = re.search(r"Banned IP list:\s*(.+)", r.stdout)
    return m.group(1).split() if m else []

def get_all_jails():
    r = subprocess.run(["fail2ban-client", "status"],
                       capture_output=True, text=True)
    m = re.search(r"Jail list:\s*(.+)", r.stdout)
    return [j.strip() for j in m.group(1).split(",") if j.strip()] if m else []

# Afficher toutes les IPs bannies
for jail in get_all_jails():
    ips = get_banned_ips(jail)
    if ips:
        print(f"[{jail}] {len(ips)} IP(s) bannies: {' '.join(ips)}")

```

— AUTO-DÉBAN D'UNE IP LÉGITIME —

```

def debannir_partout(ip):
    """Débannit une IP dans tous les jails (fail2ban >= 0.10)."""
    r = subprocess.run(["fail2ban-client", "unban", ip],
                       capture_output=True, text=True)
    print(f"Déban {ip}: {r.stdout.strip()}")

```

— SCRIPT DE RAPPORT QUOTIDIEN —

```

import sqlite3
from datetime import datetime, timedelta

def rapport_24h():
    with sqlite3.connect("/var/lib/fail2ban/fail2ban.sqlite3") as conn:
        hier = (datetime.now() - timedelta(hours=24)).timestamp()
        c = conn.cursor()
        c.execute("SELECT jail, ip, timeofban FROM bans WHERE timeofban > ? "
                  "ORDER BY timeofban DESC", (int(hier),))
        bans = c.fetchall()
        print(f"=== {len(bans)} bans dans les dernières 24h ===")
        for jail, ip, ts in bans:
            print(f" {datetime.fromtimestamp(ts):%H:%M:%S} [{jail}] {ip}")

rapport_24h()

```

— WHITELIST AUTOMATIQUE (protection contre auto-bannissement) —

```

MES_IPS = ["203.0.113.5", "192.168.1.0/24", "10.0.0.0/8"]

def appliquer_whitelist():
    for jail in get_all_jails():
        for ip in MES_IPS:
            subprocess.run(["fail2ban-client", "set", jail,
                           "addignoreip", ip], capture_output=True)
    print(f"Whitelist appliquée à {len(get_all_jails())} jail(s)")

appliquer_whitelist()

```

RÉCAPITULATIF

— INSTALLATION —

```

sudo apt install fail2ban          # Debian/Ubuntu
sudo systemctl enable --now fail2ban

```

— FICHIERS CLÉS —

/etc/fail2ban/jail.local	□ Vos jails (copier jail.conf, modifier)
/etc/fail2ban/filter.d/mon.conf	□ Vos filtres (regex <HOST>)
/etc/fail2ban/action.d/mon.conf	□ Vos actions shell
/etc/fail2ban/action.d/mon.py	□ Vos actions Python (ActionBase)
/var/lib/fail2ban/fail2ban.sqlite3	Base SQLite des bans

— COMMANDES ESSENTIELLES —

fail2ban-client ping	Serveur en vie ?
fail2ban-client status	Liste les jails
fail2ban-client status sshd	Détail du jail sshd
fail2ban-client set sshd banip IP	Bannir une IP
fail2ban-client unban IP	Débannir partout
fail2ban-client reload	Recharger la config
fail2ban-regex /var/log/auth.log sshd	Tester un filtre

— GUIDE DE DÉCISION RAPIDE —

Surveiller un service	→ jail.local [nom-jail] avec filter + logpath
Créer un filtre	→ filter.d/mon-filtre.conf avec failregex <HOST>
Tester un filtre	→ fail2ban-regex LOGFILE FILTERFILE -v
Action shell simple	→ action.d/mon-action.conf (actionban/actionunban)
Action complexe (API/BDD)	→ action.d/mon-action.py (hériter ActionBase)
Bannir manuellement	→ fail2ban-client set JAIL banip IP
Débannir manuellement	→ fail2ban-client unban IP
Script Python	→ subprocess.run(["fail2ban-client", ...])
Lire les bans en Python	→ sqlite3 /var/lib/fail2ban/fail2ban.sqlite3
Modifier config à chaud	→ fail2ban-client set JAIL maxretry N

=====